

Module 22: Multi-File Projects & Namespaces

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Every program so far has lived in one .cpp file. This module turns students into multi-file programmers: headers declare, source files define, include guards stop double inclusion, and the build is understood as two separate steps - compiling and linking. Students learn to read the two linker errors they will definitely meet (undefined reference and multiple definition), use forward declarations to break include cycles, organise code in namespaces, and build with CMake. From this module on, every Part 3 project is a multi-file CMake project, so the time invested here pays back in every later session.

Suggested Duration: 90–105 minutes

Learning Objectives

By the end of this module, students will be able to:

- Split a class into a header (.h) and a source file (.cpp), deciding correctly what goes in each.
- Protect headers with `#pragma once` and recognise classic include guards.
- Explain compiling vs linking, and build a program from several .cpp files with `g++`.
- Diagnose and fix 'undefined reference' and 'multiple definition' linker errors.
- Use forward declarations to reduce includes and break circular dependencies.
- Define and use namespaces, and explain why using namespace must never appear in a header.
- Build a small project with a `CMakeLists.txt`.

Prerequisites

- Module 21 (out-of-class member definitions with `ClassName::`).
- Part 1 Module 0 (terminal basics) - students will be typing several build commands.

Materials & Instructor Prep

- 22-multi-file-projects/examples/ - every example is a folder; open 02_header_and_source in an editor with three panes.
- CMake installed on the instructor machine (`cmake --version`). Check student machines before the session; the README lists install commands for each OS.

Key Vocabulary

Term	Definition
Header file (.h)	A file containing declarations - class definitions and function prototypes - that other files #include.
Source file (.cpp)	A file containing definitions (function and method bodies), compiled on its own into an object file.
Include guard / #pragma once	A mechanism that makes a header's contents be processed only once per .cpp file, however many times it is included.
Compiling vs linking	Compiling turns each .cpp into an object file; linking combines object files into one program, connecting each use of a function to its definition.
Forward declaration	A statement such as class Course; that tells the compiler a class exists, enough for pointers and references to it.
Namespace	A named scope that groups related names so they don't collide with names elsewhere, e.g. makerspace::energy::Appliance.
CMake	A build tool: a CMakeLists.txt describes the project once, and CMake generates and runs the build commands.

Lesson Outline & Instructor Talking Points

1. Why split a program? (10 min)

- Show 01_single_file/main.cpp and ask what happens to it at 2,000 lines with three people editing it.
- List the four benefits from the README: organisation, faster builds, teamwork, and abstraction (the header shows what, the .cpp hides how).

2. Header and source, built two ways (25 min)

- Open 02_header_and_source/ in three panes. Point out quotes vs angle brackets, the .cpp including its own header, and main.cpp never seeing a method body.
- Build with one g++ command, then again in separate -c steps, showing the .o files appear. Then deliberately leave point.cpp off the command line and read the 'undefined reference' error together.

Instructor note: Students will hit linker errors repeatedly in Parts 3's projects. Reading one calmly together now - 'no line number means the linker, so check the file list and the definitions' - saves a great deal of frustration later.

3. Include guards and multiple definitions (15 min)

- Run 03_include_guards, then delete the guard lines from component.h and show the redefinition error.
- Put a non-inline function body in a header included by two .cpp files and show 'multiple definition'. Fix it by moving the body to a .cpp file.

4. Forward declarations and namespaces (20 min)

- Walk through 05_forward_declarations: Course and Teacher refer to each other, and the forward declarations break the cycle. Ask when a forward declaration would NOT be enough (a by-value member, a base class, calling a method).
- Run 04_namespaces: two Sensor classes coexist. Connect namespace syntax to the std:: they have written since Module 1, and explain the 'never using namespace in a header' rule.

5. CMake (20 min)

- Build 06_cmake_project with `cmake -S . -B build` and `cmake --build build`. Change one .cpp file and rebuild, showing only that file recompiles.
- Stress that build/ is generated and never committed - the repository's .gitignore already handles it.

Instructor note: If CMake isn't installed on some machines, don't let it block the session: every Part 3 project README shows the equivalent plain g++ command.

Common Student Misconceptions

Misconception	How to Address It
"You list the .h files on the g++ command line too."	Headers are pasted in by #include; only .cpp files are compiled. Listing headers does nothing useful and can create confusing files.
"Undefined reference means I forgot to #include something."	A missing #include gives a compile error. 'Undefined reference' comes from the linker: the definition wasn't compiled in, was misspelled, or doesn't match the declaration.
"using namespace std; in a header saves typing for everyone."	It forces hundreds of names into every file that includes the header, causing silent clashes. Use it only locally in .cpp files, or write std::.
"Forward declarations are a hack."	They are standard practice: they reduce build times and are the normal way to break circular includes.

Exercises & Module Project

Exercise: 22-multi-file-projects/exercises/exercise1.cpp - split a working single-file Gradebook into gradebook.h, gradebook.cpp and main.cpp inside a namespace, verify identical output, then add a CMakeLists.txt. Project (three tracks): each starter is a CMake project with one class already split as a model; students split their second class (Generic - Club; EE - Room plus a shared constants header; Biomedical - Doctor using a forward declaration of Patient) and build with both CMake and g++.

Facilitation guidance: Check that every header has #pragma once, no header contains using namespace, each .cpp includes its own header first, and the build/ folder is not committed. Ask each student to explain, from their README answer, which files CMake recompiles after a change - it reveals whether they understand separate compilation.

Discussion & Comprehension Check Questions

1. Why does main.cpp compile even though it never sees the body of distanceTo()?

Guidance: Compiling only needs the declaration to check the call is valid. The linker later connects the call to the definition compiled from point.cpp.

2. When is a forward declaration not enough?

Guidance: When the compiler needs the class's size or members: a member held by value, a base class, or code that calls one of its methods.

3. Why is a function body in a header a problem, but a method body written inside a class in a header is not?

Guidance: Each .cpp that includes the header gets a copy of the function, causing 'multiple definition'. Methods defined inside a class are implicitly inline, which tells the linker duplicates are expected.

Assessment Ideas

- Linker-error clinic: give three broken mini-projects (missing .cpp, missing ClassName::, function body in a header) and ask students to diagnose each from the error message alone.
- Ask students to draw which files are recompiled after editing a header vs editing a single .cpp.

Answer Key Notes

See 22-multi-file-projects/exercises/solutions/exercise1/ (gradebook.h, gradebook.cpp, main.cpp and CMakeLists.txt).

Wrap-Up & Bridge to Next Module

Bridge forward: "Our projects are now properly organised. Next module looks at the other end of every object's life: when objects are destroyed, what destructors do, and RAII - the idea behind every resource-safe class in C++."

Instructor Tips

- Do the linker-error demonstrations live and slowly; they are the most practically valuable minutes of the module.
- Check CMake installation in the session before, not during, this module.